Parallel & Distributed Computing Lab

IMSciences

Lab Instructor: Saim Sanan

Week 3, Lab 1

OpenMP: Threads, Race, and Reduction

---

1 Ideas

- **OpenMP** — hints in C (`#pragma omp ...`), a small library, and `-fopenmp`. One process, shared memory. Not MPI. `pthread_create` starts threads by hand; OpenMP: gcc starts them.
- `#pragma omp parallel` — every thread runs that block. Same source text is not the same work unless each thread uses different data or a worksharing construct. Four identical hellos only prove the team exists.
- `#pragma omp parallel for` — gcc splits one `for` loop. The word `for` on the pragma splits `i`. `parallel` alone around a sum loop means every thread runs the *whole* loop.
- `omp_set_num_threads(4)` — team size. It overrides `export OMP_NUM_THREADS` when both are set. `omp_get_thread_num()` — who am I. `omp_get_num_threads()` — how many in the team right now (1 outside a parallel block). `omp_get_max_threads()` — cap, even in a serial program.
- **Race** — `sum += i` is read, add, write. Total is usually too small. **Reduction** — private copies, then gcc adds them. **Coherence** is hardware, not this bug.

> `main before` / `main after` print once. Hello prints once per thread. Wrong `sum` = race. Matching expected = reduction.

2 Commands

Terminal

```
nano hello_openmp.c
gcc -fopenmp hello_openmp.c -o hello
./hello

nano race.c
gcc -fopenmp race.c -o race
./race

nano reduction.c
gcc -fopenmp reduction.c -o reduction
./reduction
```

nano: Ctrl+O, Enter (save), Ctrl+X (quit).
Forgot `./` → command not found.
Forgot `-fopenmp` → error on `omp.h`, or still one thread.

3 Hello (the team)

C

```
#include <stdio.h>
#include <omp.h>
```

```
int main(void) {
    printf("main before\n");
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        printf("hello from thread %d of %d\n",
               omp_get_thread_num(),
               omp_get_num_threads());
    }

    printf("main after\n");
    return 0;
}
```

`main before` and `main after` print once. Hello prints once per thread (four times). The order of the four hellos can change (OS scheduler).

4 Race (wrong total)

Expected for `i = 0` to `N-1`:

$$\frac{N(N-1)}{2}$$

For `N = 100000`, expected = 4999950000.

C

```
#include <stdio.h>
#include <omp.h>

#define N 100000

int main(void) {
    long i;
    long sum = 0;

    #pragma omp parallel for
    for (i = 0; i < N; i++) {
        sum += i;
    }

    printf("sum = %ld\n", sum);
    printf("expected = %ld\n", (long)N * (N - 1) / 2);
    return 0;
}
```

Run three times. `sum` is usually not the expected value.

5 Reduction (correct total)

The only new clause:

C

```
#pragma omp parallel for reduction(+:sum)
```

C

```
#include <stdio.h>
#include <omp.h>
```

```
#define N 100000

int main(void) {
    long i;
    long sum = 0;

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < N; i++) {
        sum += i;
    }

    printf("sum = %ld\n", sum);
    printf("expected = %ld\n", (long)N * (N - 1) / 2);
    return 0;
}
```

The printed `sum` must equal expected on every run.